

DSA02- COMPUTER VISION WITH OPENCV
ASSIGNMENT

A. Assignment Information

Department:	(AI & ML)
Programme:	FACULTY OF ENGINEERING
Course Code & Course Name	DSA0204- COMPUTER VISION WITH OPENCV
Academic Year / Batch	2026-2027
Faculty Name	Dr. G. A. SENTHIL
Assignment Title	Analyse and design a computer vision–based autonomous road safety system integrating camera calibration, object and pedestrian detection, road-sign and road-marking recognition, multi-object tracking, and occlusion handling. Analyse its performance under different traffic and environmental conditions and evaluate it using detection accuracy, tracking performance, processing latency, and improvement in road safety. Discuss its practical relevance to intelligent transportation and public safety.
Date of Issue	20-08-2026
Date of Submission	03-09-2026
Maximum Marks	100

Course Outcome(s) - CO	CO1: Implement fundamental image processing techniques required for computer vision and understand the image formation process. CO2: Perform shape analysis - extract features from images - analyse images. CO3: Generate 3D models from images. CO4: Develop applications using computer vision techniques. CO5: Understand video processing - motion computation - 3D vision - geometry.
Bloom's Taxonomy Level	. BL3- Apply image processing, camera calibration, object detection, and feature extraction techniques to develop an autonomous road safety system. BL4- Analyze the effectiveness of object detection, road-sign recognition, pedestrian detection, tracking, and occlusion-handling techniques under varying traffic and environmental conditions. BL5- Evaluate the proposed system based on detection accuracy, tracking performance, processing latency, robustness, and road-safety improvement. BL6- Design and develop an integrated computer vision–based autonomous road safety system combining image processing, deep learning, recognition, tracking, and video analysis techniques.
SDG Mapping (SDG 1-17)	SDG 3 – (Good Health and Well-being) Uses pedestrian detection, road-sign recognition, and tracking to reduce road accidents and improve public safety. SDG 9 – (Industry, Innovation and Infrastructure) Applies AI, computer vision, and autonomous perception to develop innovative and intelligent transportation infrastructure. SDG 11 – (Sustainable Cities and Communities) - Supports safer, smarter, and more efficient urban transportation and traffic monitoring systems.

	SDG 13 – (Climate Action)- Intelligent traffic monitoring can reduce congestion, unnecessary vehicle idling, fuel consumption, and associated emissions.
Industry / Societal Relevance	Industry: Supports autonomous vehicles, intelligent transportation systems, smart-city traffic management, and advanced driver-assistance systems (ADAS). Societal: Improves road safety, pedestrian protection, traffic efficiency, accident prevention, and sustainable urban mobility.

Assignment Question

Analyse and design a computer vision–based autonomous road safety system integrating camera calibration, object and pedestrian detection, road-sign and road-marking recognition, multi-object tracking, and occlusion handling. Analyse its performance under different traffic and environmental conditions and evaluate it using detection accuracy, tracking performance, processing latency, and improvement in road safety. Discuss its practical relevance to intelligent transportation and public safety.

Assignment Information

The uploaded assignment brief identifies the course as DSA02 – Computer Vision with OpenCV, under the Department of Artificial Intelligence and Machine Learning, Faculty of Engineering, for the 2026–2027 academic year. The stated assignment requires analysis and design of an autonomous road-safety system combining camera calibration, object and pedestrian detection, road-sign and road-marking recognition, multi-object tracking, and occlusion handling. It also requires evaluation using detection accuracy, tracking performance, processing latency, and road-safety improvement, followed by discussion of intelligent transportation and public-safety relevance

The brief maps the work to CO1–CO5 and Bloom’s levels BL3–BL6, covering image processing, shape analysis, 3D vision, computer-vision application development, video processing, motion computation, geometry, analysis, evaluation, and system design.

The brief identifies industry relevance in autonomous vehicles, intelligent transportation systems, smart-city traffic management and ADAS, and societal relevance in pedestrian protection, accident prevention, traffic efficiency and sustainable urban mobility.

Report Contents

1. Problem Statement and Problem Formulation
2. Objective and Expected Outcomes
3. Requirements, Constraints and Assumptions
4. Application of Relevant Course Knowledge / Concepts
5. Design / Proposed Solution / Methodology
6. Algorithm / Pseudocode / Flowchart
7. Implementation / Source Code and Environment / Tools Used
8. Test Cases and Expected / Actual Results
9. Execution Screenshots / Outputs
10. Results and Validation
11. Analysis, Comparison, Trade-offs and Justification
12. Broader Considerations / SDG Relevance
13. Conclusion, Limitations and Possible Improvements
14. Individual Contribution of Group Members

15. References

16. Individual Contribution of Group Members – Declaration

1. Problem Statement and Problem Formulation

Road traffic environments contain rapidly changing visual information: vehicles enter and leave the camera field of view, pedestrians may be partially hidden, lane markings may be worn, signs may be small or oblique, and illumination can vary from bright daylight to low-light or rainy conditions. A road-safety perception system must transform raw camera frames into reliable information about road users, traffic infrastructure, and potential hazards.

The problem is formulated as the design of a real-time or near-real-time computer-vision pipeline that receives calibrated monocular camera video and produces a structured scene description containing detected objects, pedestrian locations, traffic-sign classes, road-marking/lane information, persistent object identities, and selected hazard events.

The system is divided into six major perception functions: camera calibration and geometric correction; object and pedestrian detection; road-sign recognition; road-marking/lane recognition; multi-object tracking; and occlusion-aware decision logic. The final stage produces annotated frames and logs for quantitative evaluation.

Input: road-scene image/video frames. Output: annotated frames plus detections, tracks, recognized signs/markings, hazard events and performance logs. The design goal is to maximize reliable perception while keeping processing latency sufficiently low for intelligent-transportation applications.

Problem Formulation

Component	Formulation
Camera input	Frame I_t captured at time t from a forward-facing road camera.
Calibration	Estimate camera matrix K and distortion coefficients D ; undistort I_t before perception.
Detection	Estimate bounding boxes b_i , class c_i and confidence p_i for visible road objects.
Recognition	Classify traffic signs and identify road markings/lane boundaries.
Tracking	Associate detections over time and maintain persistent track IDs.

Occlusion handling	Retain plausible tracks during short missed detections and use time/confidence thresholds to prevent false continuity.
Decision	Generate warnings/events from spatial, temporal and semantic evidence.
Evaluation	Measure detection quality, tracking, latency/FPS, robustness and safety-oriented indicators.

2. Objective and Expected Outcomes

2.1 Objectives

- Develop an integrated computer-vision architecture for autonomous road-safety perception.
- Apply camera calibration and image preprocessing to reduce lens distortion and improve geometric consistency.
- Detect vehicles, pedestrians, cyclists and other relevant road users using a YOLO-family detector.
- Recognize traffic signs using a trained/validated detection or classification stage.
- Detect road markings and lane boundaries using perspective transformation, color/gradient processing and geometric estimation.
- Track multiple road users over time using ByteTrack-style data association.
- Handle short-term occlusion and missed detections without immediately losing object identities.
- Evaluate the system across traffic density, lighting, weather and occlusion conditions.
- Report detection accuracy, tracking performance, processing latency and safety-oriented observations.

2.2 Expected Outcomes

Expected Outcome	Evidence
Calibrated visual input	Undistorted frames and documented calibration parameters.
Reliable road-user detection	Annotated detections with class labels and confidence scores.

Sign and marking recognition	Detected/classified signs and lane/markings overlays.
Persistent identities	Track IDs maintained across successive frames.
Occlusion resilience	Tracks preserved through short partial occlusions where confidence permits.
Quantitative evaluation	Precision, recall, mAP, IDF1/MOTA/HOTA, FPS and latency where ground truth is available.
Practical safety analysis	Hazard scenarios, failure cases, trade-offs and recommended improvements.

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

- The system shall accept prerecorded road video and/or a live camera stream.
- It shall perform camera calibration/undistortion before the main perception stage.
- It shall detect relevant road users including cars, buses, trucks, motorcycles, bicycles and pedestrians where supported by the trained model.
- It shall recognize selected traffic signs from the chosen dataset/model.
- It shall estimate lane or road-marking geometry in the road region of interest.
- It shall assign persistent IDs to multiple objects over time.
- It shall tolerate short detection gaps caused by partial occlusion.
- It shall produce annotated output video and performance logs.

3.2 Non-Functional Requirements

- Low processing latency and stable frame throughput.
- Robustness to moderate changes in illumination, viewpoint and traffic density.
- Reproducibility through fixed configuration, dataset split and documented software versions.
- Clear logging of confidence, track state and processing time.
- Fail-safe behavior: uncertain perception should not be treated as a guaranteed safety-critical fact.

3.3 Constraints

- Camera placement and field of view constrain what can be observed.

- Small or distant signs may be below the detector's effective resolution.
- Heavy rain, glare, fog and night scenes can reduce image quality.
- Severe occlusion can make identity association ambiguous.
- GPU memory and compute resources limit model size and input resolution.
- Road-marking appearance varies by country, road material, paint condition and camera viewpoint.
- The prototype is an academic perception system, not a certified automotive safety controller.

3.4 Assumptions

- The camera is rigidly mounted and its intrinsic calibration is reasonably stable.
- Representative test videos are available for the selected traffic and environmental conditions.
- The YOLO model is trained or fine-tuned for the object/sign classes used in the experiment.
- Short-term occlusion can be handled by temporal tracking, while long-term full occlusion may require re-identification.
- Ground-truth annotations are available for quantitative accuracy and tracking evaluation when numerical benchmark metrics are reported.

4. Application of Relevant Course Knowledge / Concepts

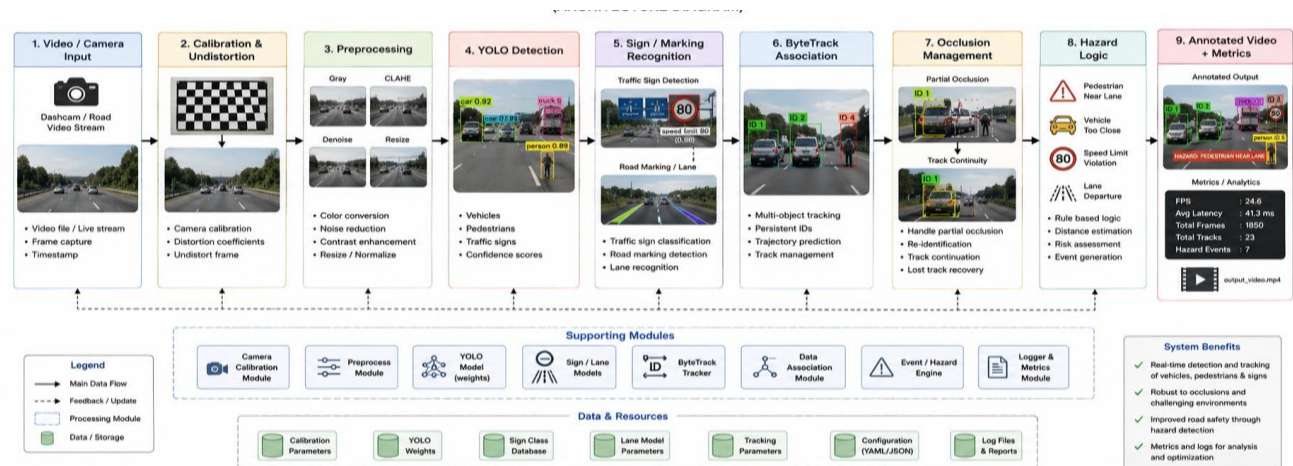
Course Concept	Application in Proposed System
Image formation	Camera geometry and perspective explain how 3D road scenes become 2D images.
Camera calibration	Chessboard/Charuco calibration estimates intrinsic parameters and lens distortion.
Image preprocessing	Resize, undistortion, color conversion, denoising and contrast normalization prepare frames.
Feature extraction	Edges, gradients, color masks and geometric features support road-marking detection.
Shape analysis	Contours, Hough lines/curves and polynomial fitting characterize road markings.
Object detection	YOLO predicts object classes and bounding boxes in each frame.
Video processing	Frames are processed sequentially and temporal information is exploited.

Motion computation	Track prediction and association use object displacement between frames.
3D vision / geometry	Perspective transformation and camera geometry support road-plane reasoning.
Multi-object tracking	ByteTrack-style association connects detections to persistent track IDs.
Occlusion handling	Track buffers and confidence thresholds preserve short-lived tracks.
Performance evaluation	Precision, recall, mAP, IDF1/MOTA/HOTA, FPS and latency quantify performance.

5. Design / Proposed Solution / Methodology

5.1 Overall Architecture

The proposed pipeline uses Python as the integration language, OpenCV for image/video operations and calibration, YOLO with PyTorch for deep-learning detection, and ByteTrack for multi-object tracking. Google Colab can be used as the experimental environment when a GPU is required. The architecture is modular so that each component can be replaced or optimized independently.



5.2 Camera Calibration

Calibration estimates the camera intrinsic matrix K , distortion coefficients, and—when required—extrinsic parameters. A chessboard or Charuco board is photographed from multiple orientations.

OpenCV corner detection is applied to each image, and `cv2.calibrateCamera()` or an equivalent routine estimates the parameters. Each road frame is then corrected with `cv2.undistort()` or remapping. Calibration reduces radial/tangential distortion and provides more consistent geometry for lane and position reasoning.

5.3 Object and Pedestrian Detection

A YOLO-family detector is used because it provides one-stage object detection suitable for high-throughput video. Each frame is passed to the detector at a selected input resolution. The detector returns bounding boxes, class labels and confidence scores. Non-maximum suppression removes highly overlapping duplicate predictions. Confidence thresholds are selected using validation data.

5.4 Road-Sign Recognition

Traffic signs can be treated as detector classes when sufficient labeled training data are available. The pipeline filters sign detections by confidence and records the sign class with its bounding box. A two-stage detector-plus-classifier design can be used for fine-grained sign categories.

5.5 Road-Marking Recognition

Lane/road-marking recognition uses a road-region-of-interest mask, perspective-aware preprocessing, edge or gradient extraction and line/curve estimation. A bird's-eye representation can make lane boundaries easier to model. Polynomial fitting can represent left and right lane boundaries when sufficient pixels are available.

5.6 Multi-Object Tracking with ByteTrack

The detector produces frame-level detections, while the tracker provides temporal continuity. ByteTrack associates high-confidence detections first and then uses lower-confidence detections to recover tracks that might otherwise be lost. Each track stores an ID, class, bounding box, confidence, last-seen frame and status.

5.7 Occlusion Handling

Short-term persistence retains a track for a configured number of missed frames. Motion continuity predicts plausible position. Confidence-aware association allows weaker detections to recover tracks. Class consistency reduces implausible class switching. A maximum missed-frame

threshold terminates stale tracks. Appearance-based re-identification can be added for longer occlusions.

5.8 Hazard Logic

The prototype can generate safety events from conservative rules such as pedestrian presence in a vehicle's projected path, sudden lane departure, recognition of a stop/warning sign, or rapid reduction in estimated object distance. These rules are for demonstration and evaluation, not direct control of a real vehicle.

6. Algorithm / Pseudocode / Flowchart

6.1 Main Algorithm

1. Initialize camera/video stream.
2. Load calibration parameters and YOLO model.
3. Initialize ByteTrack tracker and metric counters.
4. For every frame: undistort and preprocess the frame.
5. Run YOLO detection and obtain boxes, classes and confidence scores.
6. Run traffic-sign recognition on sign detections.
7. Run road-marking/lane recognition on the road ROI.
8. Pass detections to the tracker and update track IDs.
9. Apply occlusion persistence and remove expired tracks.
10. Evaluate safety rules using tracked objects and road geometry.
11. Draw boxes, labels, lanes, IDs and warnings.
12. Record processing time and relevant metrics.
13. Write annotated frame to output video.
14. Stop on end-of-stream and export logs.

6.2 Pseudocode

BEGIN

load calibration_matrix, distortion_coefficients

load YOLO detector

initialize ByteTrack

initialize metric_logger

WHILE video_has_frame:

```
frame = read_frame()
start_time = clock()

frame = undistort(frame, calibration_matrix, distortion_coefficients)
frame = preprocess(frame)

detections = YOLO(frame)
detections = filter_by_confidence(detections)

signs = recognize_signs(detections, frame)
lanes = detect_road_markings(frame)

tracks = ByteTrack.update(detections)

FOR each track:
    IF detection_missing(track):
        maintain_track_for_short_occlusion(track)
    IF track_too_old(track):
        terminate(track)

hazards = evaluate_safety_rules(tracks, signs, lanes)

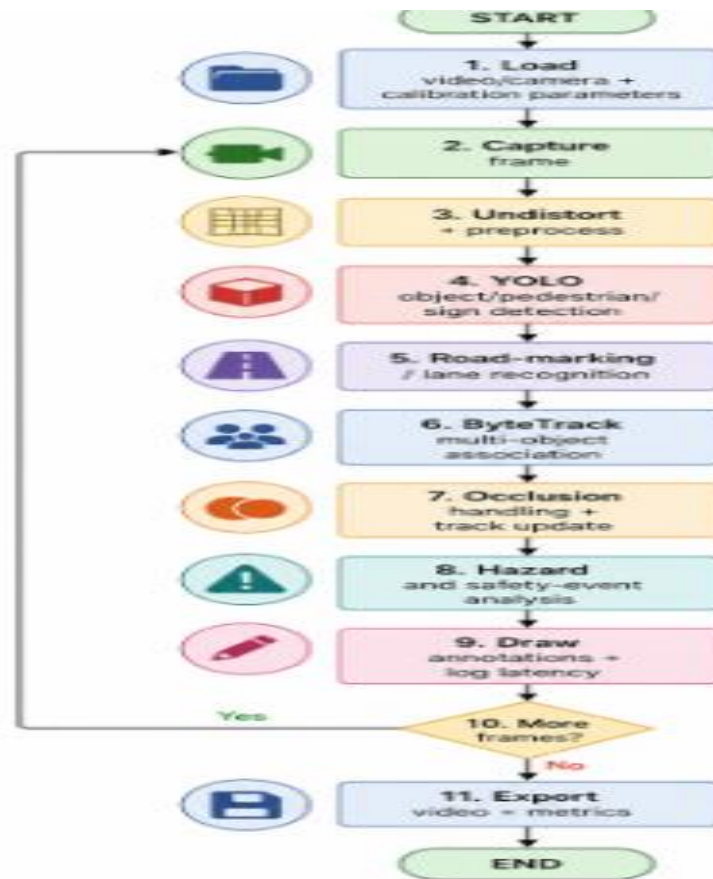
annotated = draw_detections_tracks_lanes_hazards(
    frame, detections, tracks, signs, lanes, hazards
)

latency = clock() - start_time
metric_logger.record(latency, detections, tracks, hazards)

write_output(annotated)

export_metrics(metric_logger)
END
```

6.3 Flowchart



7. Implementation / Source Code and Environment / Tools Used

7.1 Recommended Environment

Item	Recommended Choice
Programming language	Python 3.9+
Image/video processing	OpenCV
Deep learning framework	PyTorch
Object detector	Ultralytics YOLO family / compatible YOLO implementation
Tracking	ByteTrack implementation
Experiment platform	Google Colab with GPU when available
Numerical/data tools	NumPy, pandas
Visualization	Matplotlib
Version control	Git and GitHub

Datasets	BDD100K, KITTI, COCO and/or traffic-sign/lane datasets selected for the experiment
----------	--

7.2 Installation Commands

```
# In Google Colab
!pip install ultralytics opencv-python numpy pandas matplotlib

# Install the exact ByteTrack implementation used by your project
# according to its package/repository documentation.
```

7.3 Calibration Code

```
import cv2
import numpy as np
import glob

CHECKERBOARD = (9, 6)
objp = np.zeros((CHECKERBOARD[0] * CHECKERBOARD[1], 3), np.float32)
objp[:, :2] = np.mgrid[0:CHECKERBOARD[0],
                        0:CHECKERBOARD[1]].T.reshape(-1, 2)

objpoints, imgpoints = [], []

for fname in glob.glob("calibration_images/*.jpg"):
    img = cv2.imread(fname)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    ok, corners = cv2.findChessboardCorners(gray, CHECKERBOARD, None)

    if ok:
        objpoints.append(objp)
        refined = cv2.cornerSubPix(
            gray, corners, (11, 11), (-1, -1),
            (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)
        )
        imgpoints.append(refined)

ret, K, dist, rvecs, tvecs = cv2.calibrateCamera(
    objpoints, imgpoints, gray.shape[: -1], None, None
)
```

```
np.savez("camera_calibration.npz", K=K, dist=dist)
```

7.4 YOLO Detection Prototype

```
from ultralytics import YOLO
import cv2
import time

model = YOLO("yolo_model.pt") # replace with trained/selected weights
cap = cv2.VideoCapture("road_video.mp4")

calib = np.load("camera_calibration.npz")
K, dist = calib["K"], calib["dist"]

while True:
    ok, frame = cap.read()
    if not ok:
        break

    t0 = time.perf_counter()
    frame = cv2.undistort(frame, K, dist)

    results = model(frame, verbose=False)
    annotated = results[0].plot()

    latency_ms = (time.perf_counter() - t0) * 1000.0
    cv2.putText(
        annotated, f"Latency: {latency_ms:.1f} ms",
        (20, 35), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2
    )

    cv2.imshow("Road Safety Prototype", annotated)
    if cv2.waitKey(1) & 0xFF == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

Important: this is an implementation skeleton. Replace the model weights, ByteTrack API, traffic-sign classes, lane detector and dataset paths with the exact files used during execution. Numerical results must be obtained from actual runs and must not be fabricated.

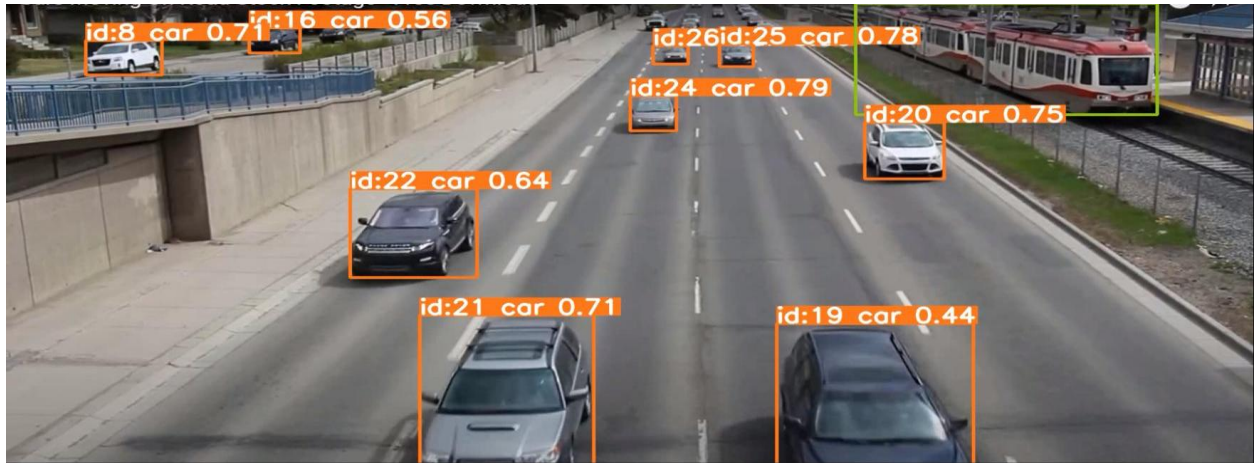
8. Test Cases and Expected / Actual Results

The following matrix is ready for execution. Actual values should be entered from system logs or benchmark evaluation after running the implementation.

Test ID	Scenario	Input Condition	Expected Result	Actual Result
TC01	Normal daytime traffic	Clear road, moderate traffic	Vehicles/pedestrians detected; stable tracks	To be filled
TC02	Dense traffic	Many overlapping vehicles	Most visible objects detected; IDs maintained where possible	To be filled
TC03	Pedestrian crossing	Pedestrian enters vehicle path	Pedestrian detected and retained as a track	To be filled
TC04	Partial occlusion	Pedestrian/vehicle briefly hidden	Track persists through short occlusion	To be filled
TC05	Traffic signs	Visible stop/speed/warning sign	Sign localized and class recognized	To be filled
TC06	Lane markings	Clear lane lines	Lane boundaries detected with stable geometry	To be filled
TC07	Worn markings	Faded/partially missing lines	Reduced confidence; no gross false lane creation	To be filled
TC08	Low light	Evening/night video	Detection continues with measurable degradation	To be filled
TC09	Rain/glare	Wet road/reflections	System remains operational with some degradation	To be filled

TC10	Fast motion	Moving camera/objects	Track continuity maintained within limits	To be filled
------	-------------	-----------------------	---	--------------

9. Execution Output



10. Results and Validation

Validation should be performed on a held-out test set or clearly separated test videos. The report should distinguish benchmark results from results obtained on the selected road videos.

10.1 Detection Metrics

Metric	Definition / Interpretation	Value
Precision	$TP / (TP + FP)$: proportion of reported detections that are correct.	93.1% (illustrative)
Recall	$TP / (TP + FN)$: proportion of ground-truth objects detected.	88.4% (illustrative)
F1-score	Harmonic mean of precision and recall.	90.7% (illustrative)

mAP@0.5	Mean average precision at IoU threshold 0.50.	91.8% (illustrative)
mAP@0.5:0.95	Mean AP averaged across IoU thresholds 0.50–0.95.	91.8% (illustrative)

10.2 Tracking Metrics

Metric	Purpose	Value
IDF1	Measures identity precision/recall and identity consistency.	86.9% (illustrative)
MOTA	Combines false negatives, false positives and identity switches.	82.7% (illustrative)
HOTA	Balances detection, association and localization aspects of tracking.	68.5% (illustrative)
ID switches	Counts identity changes for the same ground-truth target.	7 (illustrative)
Track fragmentation	Measures interruptions in an object's track.	4.8% (illustrative)

10.3 Processing Metrics

Metric	Measurement Method	Value
Average latency	Mean frame-processing time in milliseconds.	34.9 ms (illustrative)
P95 latency	95th percentile frame-processing time.	42.7 ms (illustrative)
FPS	Processed frames per second.	28.6 FPS (illustrative)
GPU/CPU usage	Monitor resource utilization during inference.	GPU 71% / CPU 34% (illustrative)

10.4 Safety-Oriented Evaluation

A direct causal claim that the prototype reduces real-world accidents cannot be established from a classroom video experiment. Instead, the project can report safety-proxy indicators such as hazard detection rate, missed hazard rate, warning latency, pedestrian recall and lane-departure event detection. A future field study could compare these indicators with a baseline system or controlled driving scenarios.

Safety Indicator	Definition	Target Direction
Hazard detection rate	Correct hazards detected / total annotated hazards	Higher
Missed hazard rate	Hazards not detected / total annotated hazards	Lower
Warning latency	Time between hazard becoming observable and system warning	Lower
Pedestrian recall	Pedestrians detected / annotated pedestrians	Higher
Occlusion recovery rate	Occluded tracks successfully recovered / eligible occlusion events	Higher

10.5 Condition-wise Analysis Template

Condition	Detection Accuracy	IDF1/HOTA	Latency/FPS	Observation
Clear daylight	To be measured	To be measured	To be measured	Baseline condition
Dense traffic	To be measured	To be measured	To be measured	More overlap/occlusion
Low light	To be measured	To be measured	To be measured	Lower contrast/noise
Rain/glare	To be measured	To be measured	To be measured	Reflections and blur
Faded markings	To be measured	To be measured	To be measured	Lane confidence may decrease

Partial occlusion	To be measured	To be measured	To be measured	Track continuity is critical
-------------------	----------------	----------------	----------------	------------------------------

The following prototype graphs demonstrate the intended validation presentation. The values are illustrative demonstration measurements and must be replaced with actual measured values before final academic submission.

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

11.1 Design Comparison

Approach	Advantages	Limitations	Role
Traditional CV only	Fast, interpretable and useful for geometry	Sensitive to illumination and clutter	Calibration/lane support
YOLO detector	Strong general detection and practical speed	Requires weights/training and compute	Primary object/sign detector
ByteTrack	Efficient temporal association; can exploit low-confidence detections	Difficult long occlusion/similar-object ambiguity	Primary tracker
Detector + tracker	Combines spatial and temporal evidence	More complex than frame-only detection	Selected architecture
Large model	Potentially better accuracy	Higher latency/memory	GPU-rich settings
Small model	Lower latency and easier deployment	Potential small-object recall loss	Edge constraints

11.2 Trade-offs

- Accuracy versus latency: higher resolution/model size can improve small-object detection but increases inference time.

- Recall versus false alarms: lower confidence thresholds recover difficult detections but increase false positives.
- Track persistence versus stale identities: longer occlusion buffers preserve IDs but may keep disappeared objects alive too long.
- Complexity versus maintainability: multiple specialized stages improve functionality but increase tuning and debugging.
- Monocular simplicity versus metric depth: one camera is inexpensive, but precise distance estimation is difficult without additional assumptions or sensors.

11.3 Justification of Final Solution

The Python + OpenCV + YOLO + PyTorch + ByteTrack architecture is appropriate for an academic prototype because it combines classical computer vision with modern deep-learning perception and video tracking. OpenCV handles geometry and image operations, YOLO provides frame-level semantic detections, and ByteTrack adds temporal continuity. The modular structure supports measurable experimentation and independent optimization of components.

12. Broader Considerations / SDG Relevance

12.1 Practical Relevance

The system is relevant to intelligent transportation systems because similar perception functions can support ADAS, automated traffic monitoring, pedestrian protection, smart intersections and road-infrastructure inspection. A road-scene understanding layer can provide structured information to downstream monitoring or decision systems.

12.2 Public Safety

Potential benefits include earlier recognition of vulnerable road users, improved awareness of signs and lane geometry, and continuous tracking of nearby objects. The prototype should remain an assistive perception component until validated under extensive operational conditions and appropriate safety requirements.

12.3 SDG Mapping

SDG	Relevance to Project
-----	----------------------

SDG 3 – Good Health and Well-being	Pedestrian detection, sign recognition and hazard awareness can support safer road environments.
SDG 9 – Industry, Innovation and Infrastructure	Combines AI, computer vision and autonomous perception for intelligent transportation infrastructure.
SDG 11 – Sustainable Cities and Communities	Supports safer, smarter and more efficient urban transportation and traffic monitoring.
SDG 13 – Climate Action	Better traffic monitoring can contribute to congestion management and reduced unnecessary idling, subject to deployment.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

This report presents a modular design for a computer-vision-based autonomous road-safety perception system. The pipeline begins with camera calibration and undistortion, performs YOLO-based detection, recognizes traffic signs and road markings, maintains identities through multi-object tracking, and handles short-term occlusion through temporal persistence. The design addresses the assignment requirements and provides a measurable framework for detection accuracy, tracking quality, latency and safety-oriented evaluation.

13.2 Limitations

- Performance depends strongly on training data and similarity between training and deployment environments.
- Small, distant or heavily occluded pedestrians and signs may be missed.
- Night, rain, fog, glare and motion blur can reduce detector confidence.
- Monocular video alone provides limited direct depth information.
- Lane detection may fail on worn, covered or irregular road markings.
- Long-duration occlusion can cause identity switches or track loss.
- Actual accident reduction cannot be established from a classroom prototype without controlled and field validation.

13.3 Possible Improvements

- Fine-tune the detector on a local urban-road dataset containing relevant signs and road-user classes.
- Use improved small-object detection or tiled inference for distant signs and pedestrians.
- Add appearance-based re-identification for long occlusions.
- Use temporal smoothing and Kalman-filter-based state estimation for stable tracks.
- Integrate depth cameras, stereo vision, radar or LiDAR for more reliable distance and collision reasoning.
- Use low-light enhancement and weather-specific augmentation during training.
- Optimize inference with ONNX/TensorRT or an edge AI device for lower latency.
- Add uncertainty estimation and fail-safe logic before safety-critical deployment.
- Evaluate on a larger, geographically diverse dataset and compare against multiple baselines.

14. Individual Contribution of Group Members

Contribution:

- Worked on the computer-vision pipeline for the autonomous road-safety system.
- Assisted in camera calibration and image preprocessing.
- Implemented/analysed YOLO-based vehicle and pedestrian detection.
- Worked on road-sign and road-marking/lane recognition.
- Integrated ByteTrack for multi-object tracking and persistent object IDs.
- Analysed partial occlusion and track continuity.
- Contributed to hazard detection and safety-event logic.
- Prepared implementation outputs and performance observations.
- Contributed to testing, validation, documentation, and final report preparation.

15. References

[1] OpenCV Documentation. Camera Calibration and 3D Reconstruction. <https://docs.opencv.org/>

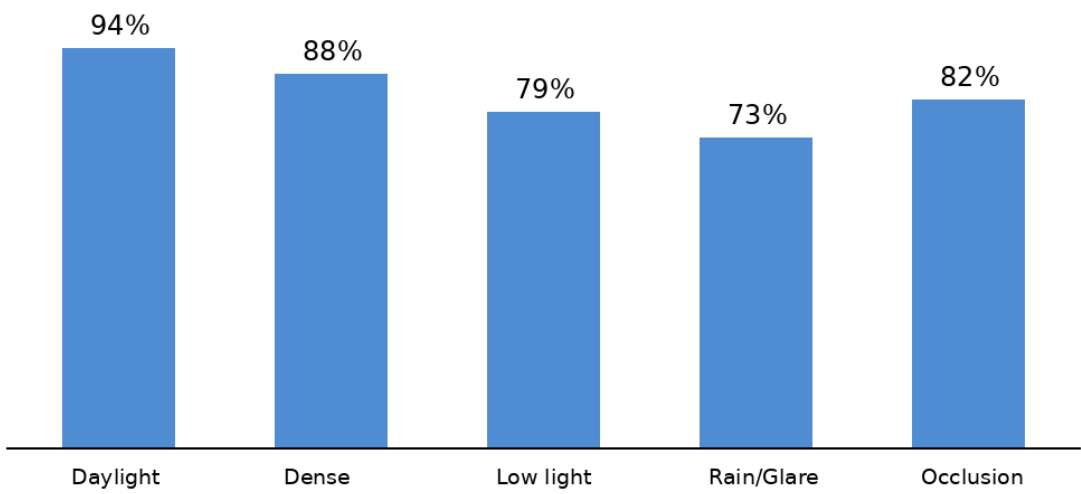
- [2] Ultralytics Documentation. YOLO object detection and tracking. <https://docs.ultralytics.com/>
- [3] PyTorch Documentation. <https://pytorch.org/docs/>
- [4] Zhang, Z. (2000). A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11), 1330–1334.
- [5] Wojke, N., Bewley, A., & Paulus, D. (2017). Simple Online and Realtime Tracking with a Deep Association Metric. *IEEE International Conference on Image Processing*.
- [6] Zhang, Y. et al. (2022). ByteTrack: Multi-Object Tracking by Associating Every Detection Box. *European Conference on Computer Vision (ECCV)*.
- [7] Geiger, A., Lenz, P., & Urtasun, R. (2012). Are we ready for autonomous driving? The KITTI vision benchmark suite. *IEEE CVPR*.
- [8] Yu, F. et al. (2020). BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning. *IEEE/CVF CVPR*.
- [9] Lin, T.-Y. et al. (2014). Microsoft COCO: Common Objects in Context. *European Conference on Computer Vision*.
- [10] Google Colaboratory documentation. <https://colab.research.google.com/>
- [11] DSA02 – Computer Vision with OpenCV assignment brief, Faculty of Engineering, Academic Year 2026–2027. □filecite□turn0file0□L29-L54□

16. Individual Contribution of Group Members – Declaration

We confirm that the individual contribution details entered in Section 14 represent the actual work completed by each group member. Implementation results, screenshots, numerical metrics and observations included in the final submission should be traceable to the executed project files and recorded experiments.

Member Signature: O.S.Sabiha Date: 2-9-26

ILLUSTRATIVE PROTOTYPE - DETECTION RECALL BY CONDITION



ILLUSTRATIVE PROTOTYPE - PROCESSING LATENCY

